AI-Based Optimization of VLSI Circuits



Under the supervision of
Dr. Aashania Antil

by

Piyush Raj, Enrollment No. 02517716023
Smriti Sinha, Enrollment No. 03317716023
Ayesha Parveen, Enrollment No. 02317716023

School of Engineering & Technology
Vivekananda Institute of Professional Studies - Technical Campus
Pitampura, Delhi - 110034

Table of Contents

Section	Page	Justification
1. Introduction (including Literature Review)	2	Defines VLSI design, the limits of the manual flow, and reviews ten works year-wise.
2. Problem Statement & Feasibility Study	5	States the problem, objectives, methodology and feasibility.
3. Hardware and Software Requirements	8	Lists the tool-chain and computing resources needed.
4. References	8	All cited literature in IEEE format.

1. Introduction

1.1 Background

Very Large Scale Integration (VLSI) is the process of realising a complete electronic system, comprising millions to billions of transistors, on a single semiconductor substrate. A modern integrated circuit is not drawn transistor by transistor; it is described behaviourally in a hardware description language such as Verilog, and is then transformed through a front-end stage of functional verification and logic synthesis, followed by a back-end stage of floorplanning, placement, clock tree synthesis, routing and sign-off analysis, into a manufacturable layout. This sequence is shown in Fig. 1.

1.2 Limitations of the Conventional Manual Flow

The structural difficulty of this flow is one of ordering rather than of tool quality. The decisions that determine power, performance and area (PPA) are taken early, at the architectural and register-transfer level (RTL) stages, where the designer selects an adder architecture, a bit width, a pipeline depth and a target clock period. Their consequences, however, are not measurable until synthesis and physical implementation have been completed. A designer comparing two adder architectures at two bit widths and two pipeline depths must therefore execute a separate tool flow for each configuration, and since each run may consume hours of runtime, exhaustive exploration is not practicable. In practice a small number of candidates are chosen from experience, and the implementation finally adopted is one that satisfies the specification rather than one demonstrated to be optimal.

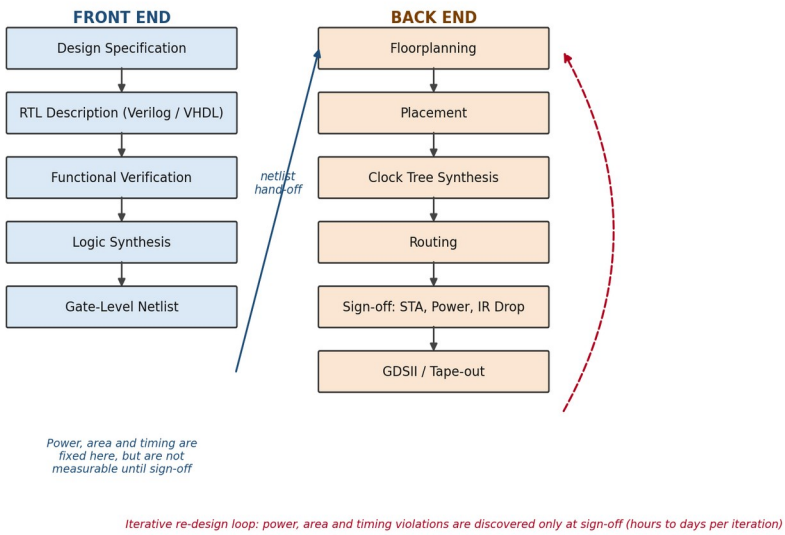


Fig. 1. Conventional VLSI design flow. Power, area and timing are fixed at RTL but are not measurable until sign-off.

1.3 Emergence of AI at the VLSI Interface

Artificial intelligence has been adopted across engineering wherever an expensive deterministic evaluation can be replaced by an inexpensive learned approximation. The VLSI flow is a favourable candidate because electronic design automation (EDA) tools generate labelled data as an ordinary by-product: every synthesis run associates a set of design parameters with the resulting power, area and timing values, and thereby yields a training example at no additional labelling cost. The role envisaged for the learned model is that of a surrogate rather than a replacement. It cannot be relied upon for sign-off, but it evaluates in milliseconds and is therefore suited to pruning a large candidate set down to a few configurations that the actual tool flow then verifies.

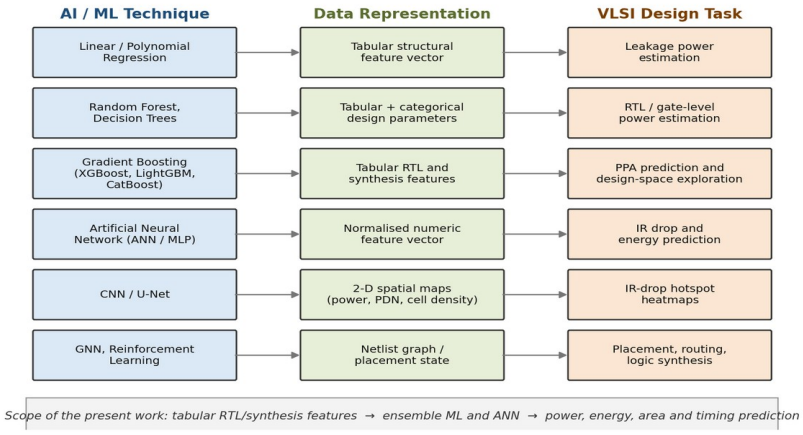


Fig. 2. Classification of AI/ML techniques by data representation and target VLSI design task.

1.4 Literature Review

The survey literature establishes the taxonomy of the field. Rapp *et al.* presented a keynote survey of machine learning for computer-aided design [1], and Amuru *et al.* traversed the full design cycle, identifying power and timing prediction as among the most mature application areas [2]. A recent survey of learning-based placement and routing reported, for a graph neural network formulation, reductions of about 3.9 percent in wirelength and 2.8 percent in power against a commercial flow [3]. All three distinguish prediction from optimisation and observe that most published work addresses the former. Fig. 2 classifies the field by learning technique, data representation and design task, and locates the scope of the present work within it.

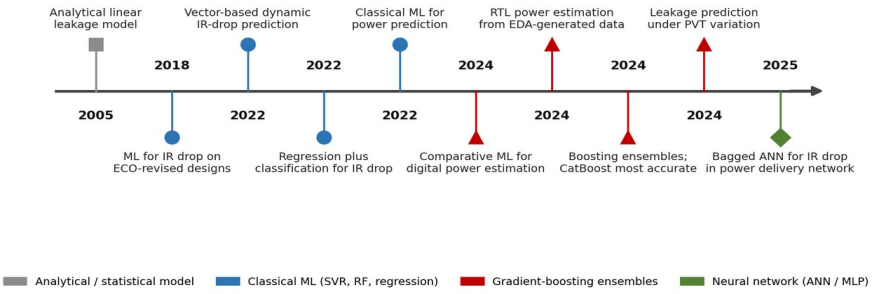


Fig. 3. Year-wise evolution of modelling approaches in the reviewed literature.

The experimental literature, summarised in Fig. 3 and Table 1, shows a clear progression. The earliest shortlisted work predates the machine-learning era: an analytical model expressed leakage power as a linear function of gate-level characteristics [4], and is retained here as the interpretable baseline against which any learned model must justify its added complexity. A second phase applied classical learning to physical-design quantities, principally IR drop on engineering-change-order revised designs [5] [6], later extended to vector-based dynamic prediction [7] and to a dual formulation treating value estimation as regression and violation detection as classification [8]. A third phase moved to power itself, first with standard regressors on tabular features [9] [10], then at register-transfer level using a corpus built by automating a commercial flow [11], and finally with gradient-boosting ensembles, among which CatBoost was reported most accurate [12]. The same techniques were applied to leakage under process, voltage and temperature variation [13]. The most recent work returns to neural modelling with a bagged artificial neural network for IR drop,

using a plain feed-forward architecture [14], which confirms that competitive accuracy on this class of problem does not require architectural complexity.

Table 1. Comparative summary of the shortlisted works, with the limitation identified in each.

Year	Ref.	Target	Technique and reported outcome	Limitation identified
2005	[4]	Leakage power	Analytical linear model on gate counts; agreement within about 5 percent.	Leakage only; cannot capture interactions between architecture, bit width and activity.
2018	[5], [6]	IR drop, ECO designs	Supervised regression on design attributes and on 17 enumerated features; average error about 3.7 mV, runtime cut up to 30 times.	Needs post-implementation features, so it cannot guide architectural choice at RTL; single metric only.
2022	[7], [8]	Dynamic and static IR drop	Regression on toggle counts and arrival times, plus classification for violating cells; error below 3 percent, speed-up about 495 times.	Requires vector-level toggle data from simulation; restricted to one physical-design quantity, so no PPA trade-off is expressed.
2022	[9]	Circuit power	Standard supervised regressors; established the tabular formulation.	Purely predictive; no mechanism recommends an improved configuration.
2024	[10]	Power, digital circuits	Comparison of four ML estimators against simulation.	Predictive only; accuracy reported on randomly partitioned rows.
2024	[11]	Power at RTL	Six best models on a dataset built by automating a Cadence flow with Python and TCL.	Single commercial tool flow; transferability to another library is not established.
2024	[12]	Circuit power	CatBoost, LightGBM, XGBoost, Random Forest; CatBoost most accurate.	Ranking established on one benchmark family; no cross-family generalisation shown.
2024	[13]	Leakage under PVT	Supervised ML on gate characterisation data; simulation effort reduced.	Confined to individual logic gates; scaling to a datapath not demonstrated.
2025	[14]	IR drop in PDN	Bagged ANN; dynamic IR-drop MSE 0.0018, coefficient of determination 0.8119.	Appreciable unexplained variance and no confidence measure per prediction.

Three observations follow. Most reviewed work is predictive rather than prescriptive, and a predictor alone does not constitute an optimisation framework. The metrics are modelled in isolation, whereas power, area and timing are mutually constrained, so a framework that cannot express the trade-off cannot recommend a configuration. Finally, accuracy is frequently reported on randomly partitioned data from a single circuit family; because configurations of one circuit are highly correlated, such a split places near-identical samples in both training and test sets and overstates generalisation. The gap addressed by this project follows from these three points.

2. Problem Statement & Feasibility Study

2.1 Problem Statement

In the conventional VLSI design flow, the power, energy, area and timing of a digital circuit cannot be determined until a complete synthesis and implementation run has been executed, so only a small number of candidate configurations can be evaluated within a practical schedule. The

problem addressed is therefore to construct a machine-learning-assisted framework that estimates these four quantities from features extracted from the RTL description, identifies violations of user-specified budgets together with the confidence attaching to each estimate, and searches the space of implementation parameters to recommend configurations offering a favourable trade-off, with the recommended candidates verified through an actual EDA flow.

2.2 Objectives

1. To build an automated pipeline that generates parameterised Verilog variants of arithmetic datapath circuits, runs an open-source synthesis and timing flow over them, and records the resulting power, energy, area and timing values.
2. To develop a feature extractor deriving gate count, flip-flop count, logic depth, fan-out, bit width, switching activity and clock period from the RTL and the synthesised netlist.
3. To train and comparatively evaluate Linear Regression, Random Forest, XGBoost, LightGBM, CatBoost and a compact artificial neural network, reporting mean absolute error, root mean squared error, the coefficient of determination and inference time.
4. To evaluate all models under a design-wise partition, in which complete circuit families are withheld from training, so that reported accuracy reflects generalisation rather than interpolation.
5. To implement a constraint and confidence layer raising power, area and timing violation flags, with a low-confidence warning from ensemble disagreement and out-of-distribution detection.
6. To implement an optimisation engine over adder architecture, bit width, pipeline depth and clock period that returns Pareto-optimal configurations, and to verify these through an actual EDA flow.

2.3 Proposed Methodology

The proposed framework is organised as the pipeline shown in Fig. 4. A Verilog description, its timing constraints and a target technology library are supplied as input. Structural features are extracted into a feature vector and presented to the trained ensemble. Predicted values are tested against the user budgets and the confidence criteria, after which the optimisation engine searches the parameter space and returns a ranked set of candidates. Only these are submitted to the full

EDA flow, and the verified results are appended to the dataset so the model may be retrained. The benchmark is developed in three phases: 8-bit and 16-bit ripple-carry, carry-lookahead and carry-select adders; then an arithmetic logic unit and a multiply-accumulate unit; and finally dissimilar designs such as a comparator and an FIR filter, reserved exclusively for testing in order to measure cross-family generalisation.

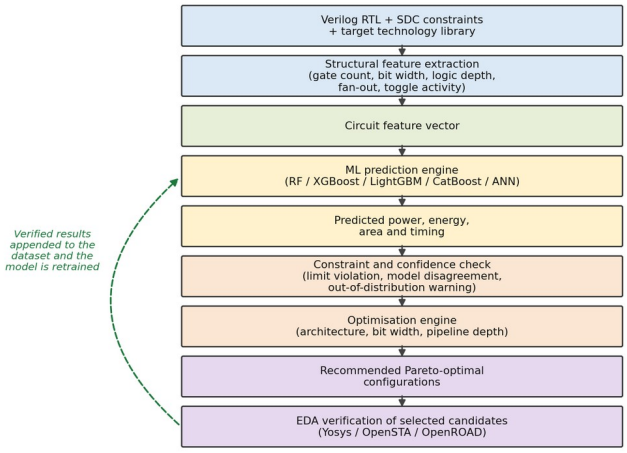


Fig. 4. Proposed machine-learning-assisted RTL design-space exploration and optimisation framework.

2.4 Feasibility Study

The project is technically feasible using established open software. Synthesis and structural extraction use Yosys, static timing analysis uses OpenSTA, and physical-design experiments use the OpenROAD flow, all accepting standard RTL, SDC, Liberty and LEF inputs. The learning components use scikit-learn, XGBoost, LightGBM, CatBoost and a compact network in TensorFlow or PyTorch, with Optuna or NSGA-II for the optimiser. The proposed network, two hidden layers of 64 and 32 units with rectified linear activation, is deliberately modest. Dataset generation is the principal effort and the principal risk: the target corpus of roughly five hundred to two thousand samples is produced by script rather than by hand, so its cost is dominated by tool runtime, and should physical-implementation runtime prove prohibitive the scope may be restricted to synthesis-level PPA without invalidating the framework. The framework is not claimed valid for arbitrary circuits, so the system reports the similarity of an input design to its training domain and warns when a design falls outside it.

3. Hardware and Software Requirements

3.1 Hardware Requirements

Resource	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher; four cores minimum, eight recommended for parallel synthesis runs
Memory	16 GB RAM recommended for physical-design experiments on the larger benchmark circuits
Storage	50 GB free space for tool installations, generated netlists, reports and the accumulated dataset
Graphics	A CUDA-capable GPU is optional only; the dataset is tabular and the network compact, so training is feasible on CPU

3.2 Software Requirements

Category	Tools
Operating system	Ubuntu 22.04 LTS or later, or Windows 10/11 with Windows Subsystem for Linux 2
EDA tools	Yosys, OpenSTA, OpenROAD / OpenLane; Cadence or Synopsys tools under academic licence where available
Technology libraries	SkyWater Sky130 and ASAP7 open process design kits
Simulation	Icarus Verilog or Verilator for functional verification and switching-activity generation
Languages	Python 3.10 or later, Verilog, and TCL for EDA tool scripting
Machine learning	scikit-learn, XGBoost, LightGBM, CatBoost, TensorFlow or PyTorch
Optimisation and data	Optuna, pymoo, NumPy, pandas, Matplotlib; Streamlit for the dashboard; Git for version control

4. References

[1] M. Rapp, H. Amrouch, Y. Lin, B. Yu, D. Z. Pan, M. Wolf and J. Henkel, "MLCAD: A survey of research in machine learning for CAD keynote paper," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 41, no. 10, pp. 3162-3181, 2022.

[2] D. Amuru, A. Zahra, H. V. Vudumula, P. K. Cherupally, S. R. Gurram, A. Ahmad and Z. Abbas, "AI/ML algorithms and applications in VLSI design and technology," Integration, the VLSI Journal, vol. 93, p. 102048, 2023.

[3] "A survey on learning-based PnR optimization for VLSI physical design automation," IEEE Access, 2025.

[4] "A precise model for leakage power estimation in VLSI circuits," in Proceedings of the IEEE International Conference, 2005.

[5] "IR drop prediction of ECO-revised circuits using machine learning," in Proceedings of the IEEE VLSI Test Symposium (VTS), 2018.

[6] Y.-C. Fang, H.-Y. Lin, M.-Y. Sui, C.-M. Li and E. J.-W. Fang, "Machine-learning-based dynamic IR drop prediction for ECO," in Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2018.

[7] "Vector-based dynamic IR-drop prediction using machine learning," in Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC), 2022.

[8] "MLIR: Machine learning based IR drop prediction on ECO revised design for faster convergence," in Proceedings of the IEEE International Conference, 2022.

[9] "Power prediction of VLSI circuits using machine learning," Computers, Materials and Continua (CMC), vol. 74, no. 1, 2022.

[10] "Power estimation of digital VLSI circuits using machine learning," in Proceedings of the IEEE International Conference, 2024.

[11] "Application of machine learning approaches for power estimation of digital VLSI circuits at register transfer level," in Proceedings of the IEEE International Conference, 2024.

[12] "Power estimation in VLSI circuits using supervised machine learning," in Proceedings of the IEEE International Conference, 2024.

[13] "Machine learning-based leakage prediction in analog logic gates under PVT variations," in Proceedings of the IEEE International Conference, 2024.

[14] "Artificial neural network based prediction model for IR drop measurement in a VLSI power delivery network," Journal of Electronic Testing: Theory and Applications, 2025.